Mathematical Programming MATH20014: Group projects

September 29, 2025

Outline of the group projects

The group projects for MATH20014 will be organised as follows. From the set of potential projects detailed below, each of you will choose a favourite project and backup projects in decreasing order of preference, according to their interest to you. The projects are all of a similar level of difficulty, but focus on different areas of mathematics, including mechanics, statistical mechanics, combinatorics, cryptography, number theory and knot theory.

Each group will then consist of four or five students who have shown interest in the same project; the groups will be assembled by the team of lecturers.

Format of the group projects

We expect the group project to take the form of a single jupyter notebook, similar to the lectures and tutorials. Your jupyter notebook will contain the different pieces of code, and the same jupyter notebook will also need to contain explanations, formulas, graphs, context and references for your project.

As you have seen in the lectures and tutorials, jupyter notebooks understand latex, so typesetting equations is straightforward. On the other hand, you may also want to embed or run other forms of content in your notebook. To do this you may also create and submit—alongside your notebook file—other types of file, including the following.

- Python code (.py) files with, for example, modules or classes that you have created, or scripts (for example scripts that carry out the time-consuming calculations or animations required by some projects).
- High-resolution images and movies obtained as output from your programs/functions.

Assessment criteria

The projects presented here contain a core part (marked **[core]** or **(core)** below) which you need to satisfactorily complete to obtain a pass mark. The other parts are extensions that you can pick and choose. The grading criteria fall under headings 1-5 below. A detailed outline can be found in the file `assessment_criteria.pdf` on Blackboard (in the same folder as the present file).

1. **Correctness.**

   (Does your code do what it is supposed to do?)

2. **Quality: structure and design.**

   (Is the structure of your code clear and easy to maintain and is it user friendly?)

3. **Documentation and Clarity.**

   (Is your code written in such a way that the interested programmer can easily understand what is going on?)

4. **Scope.**

   (How well have you developed the various parts of the project including at least one of the extensions/non core parts.)

5. **Project Writing.**

   (How good is the surrounding text of the project itself?)

Peer assessment and project submission

There will also be a peer assessment of your group where you rate the commitment and performance or your group members. This will take a similar form to the group projects in Mathematical Investigations in year 1. **The projects and peer assessments are due at 12 Noon Wednesday 4th December.**

How to collaborate on a computing project

The first step is to make sure that you have a shared drive that is accessible to all members of your team, for example by creating a shared folder on Onedrive. Then create your Jupyter notebook for the project, and edit it. A key issue is **version control**, i.e. that there is only one current version of the project (and not, say, five different branches that five different people are working on). At the same time, you will want to keep the previous versions of the notebook, so that potentially important pieces are not overwritten or lost and you can always undo changes that were a mistake.

The simplest (though not the most efficient) form of version control is to save a copy of the Jupyter notebook with the current date and your name each time you make an edit, and to always, always start from the most recently modified file.

A second very important point: the projects generally start from a basic idea and then develop this through several stages. It is crucial that you do the beginning pieces first. It is also crucial that you work on **one** version of it **together**. For example, to figure out energy conservation in planetary motion, you need to have a working integrator of the four ODEs first, etc. Please do not try to develop pieces of the project individually and then merge them, as this is almost impossible to do due people's different choices for the structure of the code. You should begin by having an online group meeting where you work through the project and set up the code structure, followed by regular meetings.

Setting up python on your own computer.

For collaboration using a shared folder on oneDrive you will need to use Python and Jupyter either on a workstation in the Computer Lab or on your own computer. In order to use your own computer for this we recommend that you install the appropriate version of the Anaconda package manager from: https://docs.anaconda.com/anaconda/install/ . Once set up, run Anaconda Navigator. From there launch Jupyter Notebook of Jupyter Lab.

Alternative ways of collaborating on a computing project

- **GitHub** If you are keen to explore more formal options for working on a coding project as a team, please consider using the **git software**, and optionally the github online code sharing platform (https://github.com/).

- **Noteable** provides an option for collaborative editing. Instructions for this can be found here: https://noteable.edina.ac.uk/documentation/collab-editing/. However (as noted in the instructions) "When inviting someone to collaboratively edit a notebook, you are bypassing all other login/authentication processes, and inviting them to access your personal workspace." Moreover access to the shared file(s) is revoked when the person sharing shuts down their server. So, although this is an option we do not recommend using Noteable for collaboration at present.

6 Code Breaking with Statistical Physics

In the appendix there are several secret messages for you to decrypt, in roughly increasing order of difficulty. Each message is written using the 26 letters of the alphabet together with spaces. (There are no other symbols such as punctuation). Each message is encrypted using a simple substitution cipher; that is, a certain (secret) permutation of the 27 characters has been applied to each character of the message in turn. Your job is to crack the code and discover the message. Ideally your final answers you should include your best guess at the correctly punctuated and cased message, as well as the raw decrypted version. Of course, the last part will require human involvement.

The provided messages are there to motivate and focus you, but are not necessarily the final destination. The broader goal is to investigate decryption methods.

For your convenience, a notebook containing the messages as strings will be provided on the Blackboard page.

1. (**core**) In the first message, the cipher is simply a cyclic shift of the 27 characters. E.g. a shift of 2 would map $A \mapsto C$, $B \mapsto D$, $\ldots$, $Y \mapsto$ space, $Z \mapsto A$, space $\mapsto B$. Find the shift and decode the message. One simple approach is to try all possibilities and check by hand which one produces readable text.

2. (**core**) The other messages each use an arbitrary permutation of the characters, so trying all the possibilities is no longer practical. Instead we will make use of statistical properties of text. In preparation for this, find a large body of English text (the larger the better) to download and analyse. Replace all lower case letters with upper case, and find a way to get rid of extra characters. One simple approach is to replace every character that is not a capital letter or a space with a space, and then replace multiple consecutive spaces with a single space. More nuanced schemes are possible - for instance, apostrophes and hyphens could be treated differently.

   For example, the following code will download the text of the novel Moby Dick and store it in a single very long string.

   ```
   import requests
   url='http://www.gutenberg.org/files/2701/2701-0.txt'
   moby=requests.get(url).text
   ```

   To avoid repeatedly spamming someone's server, it is best to save the text to your own machine and access it from there:

   ```
   with open('C:/data/moby.txt','w',errors='ignore') as f:
       f.write(moby)   # write to file


   with open('C:/data/moby.txt','r',errors='ignore') as f:
       moby=f.read()   # read from file
   ```

3. (**core**) One simple approach is to use character frequencies. In the sample English text, count the frequency with which each of the 27 characters appears, and sort them by frequency. Do the same for the coded message, and try the cipher in which the kth most common character in the message maps to the kth most common letter in English, for each k.

4. (**core**) The above may not be sufficient to get the full correct answer, but for the longer messages it should be close. Write a program to make it easy for you to try swapping the roles of any two characters (of your choice) and immediately see the effect of the decoded message. Then, starting from the result above, adjust the cipher by hand until is readable.

5. (**core**) The above method is less likely to work for the later messages, because they are not long enough to get accurate frequency counts (and some might involve unusual language). Instead use the following approach based on ideas from statistical physics. In the sample text, record the frequency (i.e. number of occurrences) of each of the 27^2 bigrams, i.e. pairs of consecutive characters such as SH, as well as the 27 characters. For each pair of characters i, j compute

$$p(i,j) := \frac{\text{frequency}(ij)+1}{\text{frequency}(i)}.$$

This represents the probability that i is followed by j; the $+1$ is a 'fudge factor' to avoid zeros. To any potential decoded message $m = i_1 i_2 \dots i_n$ we assign a score

$$S(m) = \sum_{j=1}^{n-1} \log p(i_j, i_{j+1}),$$

which measures how plausible m is as a piece of English text (it represents the logarithm of the probability of seeing m under a certain "Markov model"). Write a function that computes $S(m)$ for any message m.

6. (**core**) Implement this method, called the Metropolis algorithm (an example of a Monte Carlo method). Start with any guess m at the decrypted message, which might simply be the encrypted message itself or the result of applying a randomly chosen cipher. Now repeatedly do the following.

   Choose two characters at random and swap their roles in the cipher, to get a new candidate message m'. If $S(m') > S(m)$ then replace m with m'. If $S(m') \leq S(m)$ then replace m with m' with probability

$$\exp \frac{S(m') - S(m)}{T},$$

   otherwise just keep m.

   Repeat this for many steps (e.g. 100000 or 1000000).

   Here $T > 0$ is a parameter, which represents temperature in a physics analogy, and indicates how likely the algorithm is to accept steps that make the score worse (in hopes of finding a route making it better later). You may need to experiment to find the value of T that works best. Start by trying $T = 1$. One approach (called simulated annealing) is to start with T large (say 10), and gradually reduce it as the algorithm proceeds, ending at something small (say 0.1), perhaps by reducing it by a constant ratio at each step.

   It is useful to print the current guess m every so often (e.g. every 10000 steps), and monitor its progress.

   It may be useful to try running the algorithm multiple times, from different starting points, and to adjust the parameters, and to make manual adjustments to the cipher at the end (as before).

Possible directions for extensions:

You should focus on one or possibly two extension directions. A good project will typically involve work on extensions roughly equal to that needed to thoroughly address the core.

Quality is more important than quantity. An in-depth exploration of a few directions is typically preferable to a superficial treatment of large number.

Other directions besides those suggested are possible. If you have ideas, it is advisable to discuss them with your supervisor.

For the extensions below, besides the messages provided, you will likely find it helpful to produce your own coded messages.

7. How can the method be improved? The final few messages will require thinking outside the box. Does it help to consider trigrams, or just common ones, or whole words? What if the message is in a unknown language? Can the language be detected automatically? Can the method cope with small errors (typos) in the message, or even correct them?

8. For the simple frequency analysis approach in steps 3-4, how much adjustment is typically needed, and how can that be quantified systematically? Can you represent the required permutation graphically? How does the answer depend on the length of the message or other factors? Are some letters more problematic than others? Can you derive a probabilistic model of what is going on?

9. Can you analyse the progress of the Metropolis algorithm over time, perhaps by plotting graphs of some quantities? How do the results depend on the length, or other factors? Can this help with adjusting T, and choosing an appropriate number of steps to run for?

10. Can the Metropolis method be adapted to other ciphers such as transposition ciphers, the Vigenere cipher, multiple-substitution ciphers (perhaps using a key that is itself a piece of English text), or a one-time pad that itself consists of English text? You would need to look up what these are (e.g. on Wikipedia), and write your own programs for encrypting as well as decrypting, so that you have ciphertexts to work with.

Coded messages

1. GPKZFER JRSER EKWIGIWKWVRZ YZRCWMWCRSEVRYWEWISCRGLIGFJWRGIFYISDD
   EYRCSEYLSYWRGPKZFEJRVWJ YERGZ CFJFGZPRWDGZSJ QWJRUFVWRIWSVST
   C KPRN KZR KJREFKSTCWRLJWRFXRJ YE X USEKR EVWEKSK FER
   KJRCSEYLSYWRUFEJKILUKJRSEVRFTAWUKRFI WEKWVRSGGIFSUZRS
   DRKFRZWCGRGIFYISDDWIJRNI KWRUCWSIRCFY
   USCRUFVWRXFIRJDSCCRSEVRCSIYWRJUSCWRGIFAWUKJ

2. RECFV KUWE VJCRWQFCRCFICYWEZRWKLVJWQF SCRRZ ALVWJLAACFWNATZVW
   NFWIZRZT FWELYWVCSTWNRWLVTE NBEWZTWHLRWCLRMWS FWJCWHE WUACHWEZJWR
   WHCVVWT WRCCWTELTWECWHLRWQF S NAYVMWCPKZTCYWTECWJ JCATWTELTWEZVT
   AWKNGZTTWRWGF LYWGLKUWELYWYZRLQQCLFCYWTEF NBEWTECWY FWJMWK
   JFLYCWFNRECYWT WTECWTLGVCWVLZYW NTWLVVWTECWRVZQRW SWQLQCFWK
   ATLZAZABWYLAKZABWJCAWZAWSF ATW SWEZJWLAYWTEFCHWEZJRCVSWZAT
   WLAWZATFZKLTCWLAYWCVLG FLTCWKLVKNVLTZ AWS FWTH WE NFRWZWHLTKECYWEZJWLRWECWK
   ICFCYWRECCTWLSTCFWRECCTW SWQLQCFWHZTEWSZBNFCRWLAYWVCTTCFRWR WK JQVCTCVMWLGR
   FGCYWZAWEZRWTLRUWTELTWECWELYWCIZYCATVMWS FB TTCAWJMWQFCRCAKCWR
   JCTZJCRWECWHLRWJLUZABWQF BFCRRWLAYWHEZRTVCYWLAYWRLABWLTWEZRWH FUWR
   JCTZJCRWECWHLRWQNDDVCYWLAYWH NVYWRZTWS FWV ABWRQCVVRWHZTEWLWSNFF
   HCYWGF HWLAYWLWILKLATWCMCWSZALVVMWECWRQFLABWSF JWEZRWKELZFWHZTEWLWKFMW
   SWRLTZRSLKTZ AWLAYWHLVUCYWNQWLAYWY HAWTECWF JWFNGGZABWEZRWELAYRWT
   BCTECFWTECAWECWHF TCWLWV ABWTCVCBFLJWNQ AWLWKLGVCWS FJWZSWJMWLARHCFWT
   WTEZRWZRWLRWZWE QCWM NWHZVVWELICWLWICFMWQFCTTMWKLRCWT WLYYWT WM
   NFWK VVCKTZ AWHLTR AWRLZYWECWZWCPQCKTWTELTWHCWRELVVWGCWLGVCWT
   WB WY HAWT WA FS VUWT J FF HWLAYWT WTLUCW NFWSFZCAYWR
   JCWICFMWYCSZAZTCWACHRWLRWT WTECWRCKFCTW SWEZRWLAA MLAKCWZWK
   ASCRRWTELTWZWHLRWSZVVCYWHZTEWKNFZ RZTMWGNTWZWHLRWLHLFCWTELTWE
   VJCRWVZUCYWT WJLUCWEZRWYZRKV RNFCRWLTWEZRW HAWTZJCWLAYWZAWEZRW
   HAWHLMWR WZWHLZTCYWNATZVWZTWRE NVYWRNZTWEZJWT WTLUCWJCWZAT WEZRWK
   ASZYCAKCWGNTWTECFCWHLRWLWYCVLMWZAWTELTWLARHCFZABWTCVCBFLJWLAYWTH WYLMRW
   SWZJQLTZCAKCWS VV HCYWYNFZABWHEZKEWE VJCRWQFZKUCYWNQWEZRWCLFRWLTWCICFMWFZABW
   SWTECWGCVVW AWTECWCICAZABW SWTECWRCK AYWTECFCWKLJCWLWVCTTCFWSF
   JWEZVT AWKNGZTTWLVVWHLRWXNZCTWHZTEWEZJWRLICWTELTWLWV ABWZARKFZQTZ
   AWELYWLQQCLFCYWTELTWJ FAZABWNQ AWTECWQCYCRTLVW SWTECWRNAYZLVWECWZAKV
   RCYWLWK QMW SWZTWHEZKEWZRWECFCWFCQF YNKCYWCVRZCWQFCQLFCWT
   WJCCTWTEMWB YWE VJCRWGCATW ICFWTEZRWBF TCRXNCWSFZCDCWS FWR
   JCWJZANTCRWLAYWTECAWRNYYCAVMWRQFLABWT WEZRWSCCTWHZTEWLAWCPKVLJLTZ AW
   SWRNFQFZRCWLAYWYZRJLMWEZRWSLKCWHLRWELBBLFYWHZTEWLAPZCTM

3. TULOMREMAEKBLRSWCTMWBIHB BTN KKUT NBTULOMRB DWBKSYMBYSWMRDBTULOMRKBJUNNBRMQM
   NBKE EUKEUT NBUDPSRY EUSDB ISCEBEOMBLN UDEMAEB DWBEO
   EBUDPSRY EUSDBT DBSPEMDBIMBCKMWBESBIRM XBEOMBTULOMRB
   PEMRBEOMBWUKTSQMRHBSPBPRMVCMDTHB D NHKUKBIHBEOMB R IBY EOMY EUTU DB
   DWBLSNHY EOB NBXUDWUB NKSBXDSJDB KB NXUDWCKBUDBEOMBDUDEOBTMDECRHBDM
   RNHB NNBKCTOBTULOMRKBTSCNWBIMBIRSXMDBIHB DBUDPSRYMWB EE TXMRBKCTOBTN
   KKUT NBTULOMRKBKEUNNBMDGSHBLSLCN RUEHBESW HBEOSCFOBYSKENHB KBLCZZNMKB
   NBXUDWUBJRSEMB BISSXBSDBTRHLESFR LOHBMDEUENMWBRUK N OBPUBUKEUXOR
   GB NBYC YY BY DCKTRULEBPSRBEOMBWMTULOMRUDFBTRHLESFR LOUTBYMKK
   FMKBJOUTOBWMKTRUIMWBEOMBPURKEBXDSJDBCKMBSPBPRMVCMDTHB D NHKUKB DWBTRHLE
   D NHKUKBEMTODUVCMKB DBUYLSRE DEBTSDERUICEUSDBSPBUIDB WN DBJ KBSDBK
   YLNMBKUZMBPSRBCKMBSPBPRMVCMDTHB D NHKUK

4. RJLHXTNERXWZEWZSXLESNIFWX TEKJAAS NYSVEWZSECYWKZEKJTVYAW
   TWVERSLSELYTTXTNETYPDSLVEDJVH AXVVELJASER VED VXK AAIEWJECJEWZSEK
   AKYA WXJTVEWZSEVYPVEDSLCJRVHXEV XCEVJERZSTEWZSEVWSLTEK PSEULSSEJTEPJTC
   IEPJLTXTNERSEK AKYA WSCEWZ WERSEVZJYACEASWEWJTTSVEJUER WSLED AA VWEXTE
   WEWZSELS LEJUEWZSEOSVVSAEWJEFYVZEWZSEVWSLTECJRTE TCEAXUWEWZSEDJR

5. IZWLXSZF QEFLZTQUVLSZMIHZCIVLUZEDDZIUPZXLYAHLPZCQZLICZECZMIHZC QAO CZ
   QMLWLXZC ICZECHZIBBLCECLZTEO CZKLZHCETADICLPZKSZIZBEULIBBDL

6. RVRRKZHKTVYPJCGVCHGVGXECUKRKZGRSKZKWGVGRSPKAGVCHGVGHZVLJCAYOGRZET
   KRKZGSKWGTOGSKZJGAVPZOGHVCHOGRPQUYPCLGRSKGAVCQOGJAGSPWGYVHOGAZPKCHGRSKGCOT
   SGPCGOKYYJMGQVCGMKGWKKGRSKGTVWRKZGWRZJUKGMSVRGVGNEVKZKGAKYYJM

7. LOJ WUZCWKCTFHHFHTWUZFHQWMZEUW EAWKCWUZCWCHV

8. EYENXRPSURPGUUIRKSUGURSHYY NXRCUUIREPRCHBIRENBRMAOKUGIRKUGURS
   NX NXROREHABRFEAUBONRBGUKROHPRSURBGONURENBRPORSUGRJ JURKEIRI
   NX NXRORPKEIRJ CGOFSRIENXRIPGEPSIJUWIRENBRGUUAIRISURB
   GABRPSUYREMMRMHRFAUEGAWRORKSUNRPSUGURFEYRERWUAARORMOGU
   XNRIVHUUAIRPSEPRBENXRSUGRPEJIEAPUUG URO

9. WDYRDYLDQCSLR KTYDPYZ LXSKTYWDYQ U KTYZOITYGIDYCDYJSILPTYLDUDKOTYCIQYKQTP
   QPYODYPDRZTYCDQPDQCLDYODTYJL GIDRDQPTYSLH
   QKGIDTYCDTYMSKTDLKDTYCSIULKLYODTYADINYZSILYXKNDLYODYB
   ODKCSTJSZDYCDYOSMTJILKPDYCDYHSYPDLYHLYJDYIQDYOIDILYRSRDQP
   QYDYCDYJSQTJKDQJDYODYTSRRDKOYSIYDP KDQPYZOSQHYTYODTYRDIMODTYO
   YJF RMLDYODYPSIPYCSQPYWDYQDP KTYGIIQDYZDPKPDYZ LPKDYDPY
   YOKQTDQTKMKOKPDYCIGIDOYWDYLDPSILQ KTYUKPDYRIQKL

10. GYYUXPUXGTBQTDFU

11. ZVVFEKTFZKKO

12. RIZOGS BXTPK YWH JQNE LUC MADFV